Advanced Java Programming

Module 2

Java Beans

Outline

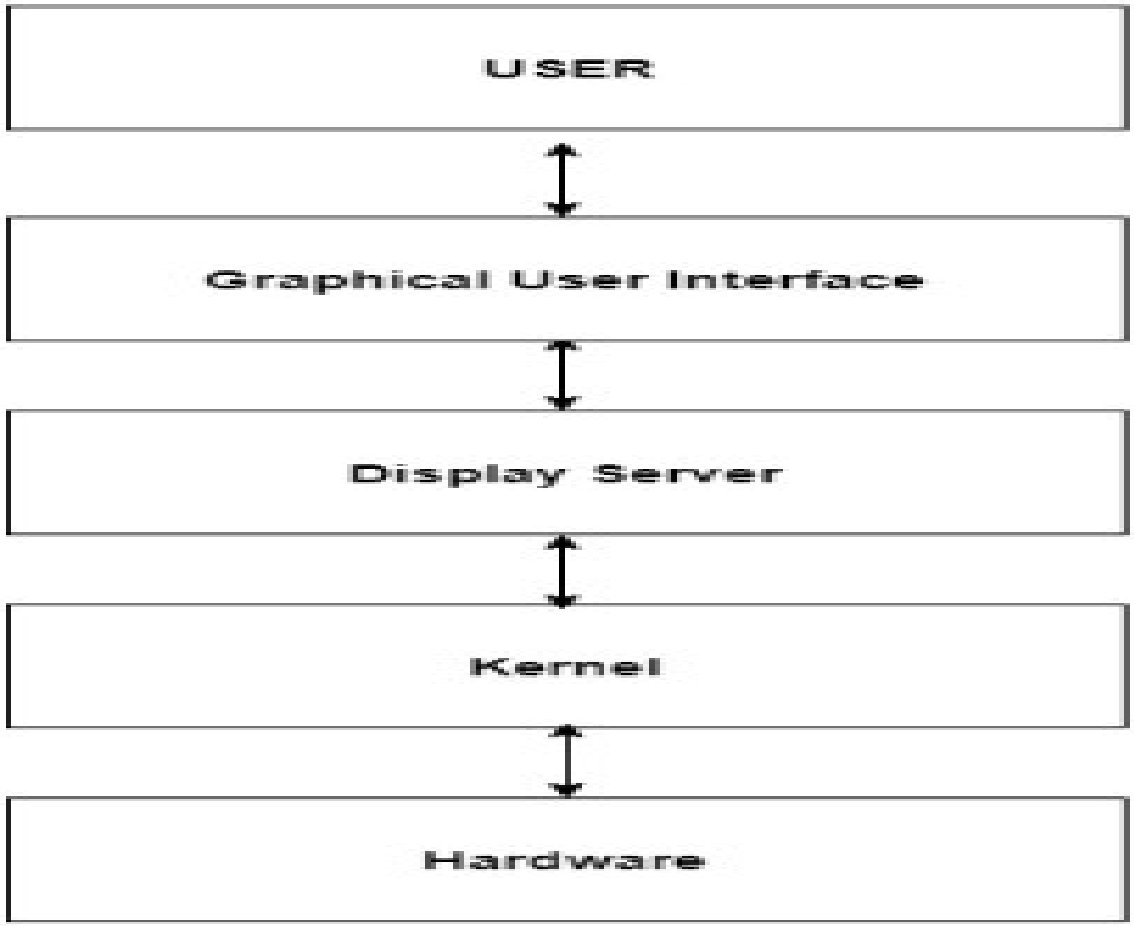
- Graphical User Interface (GUI)
- JFC
- JavaBeans
- Handling Events in JavaBeans

Graphical User Interface (GUI)

- GUI is an interface that allows users to interact with different electronic devices using icons and other visual indicators. The graphical user interfaces were created because command line interfaces were quite complicated and it was difficult to learn all the commands in it.
- In today's times, graphical user interfaces are used in many devices such as mobiles, MP3 players, gaming devices, smartphones etc.

Graphical User Interface (GUI)

- The below diagram provides the position of the graphical user interface with respect to the computer system –



Elements in Graphical User Interface

•Graphical User Interface makes use of visual elements mostly. These elements define the appearance of the GUI. Some of these are described in detail as follows −

Window

•This is the element that displays the information on the screen. It is very easy to manipulate a window. It can be opened or closed with the click of an icon. Moreover, it can be moved to any area by dragging it around.In a multitasking environment, multiple windows can be open at the same time, all of them performing different tasks.

•There are multiple types of windows in a graphical user interface, such as container window, browser window, text terminal window, child window, message window etc.

Elements in Graphical User Interface

Menu

•A menu contains a list a choices and it allows users to select one from them. A menu bar is displayed horizontally across the screen such as **pull down** menu. When any option is clicked in this menu, then the pull down menu appears.

•Another type of menu is the **context menu** that appears only when the user performs a specific action. An example of this is pressing the right mouse button. When this is done, a menu will appear under the cursor.

Icons

•Files, programs, web pages etc. can be represented using a small picture in a graphical user interface. This picture is known as an icon. Using an icon is a fast way to open documents, run programs etc. because clicking on them yields instant access.

Elements in Graphical User Interface

Controls

•Information in an application can be directly read or influences using the graphical control elements. These are also known as **widgets**. Normally, widgets are used to display lists of similar items, navigate the system using links, tabs etc. and manipulating data using check boxes, radio boxes etc.

Tabs

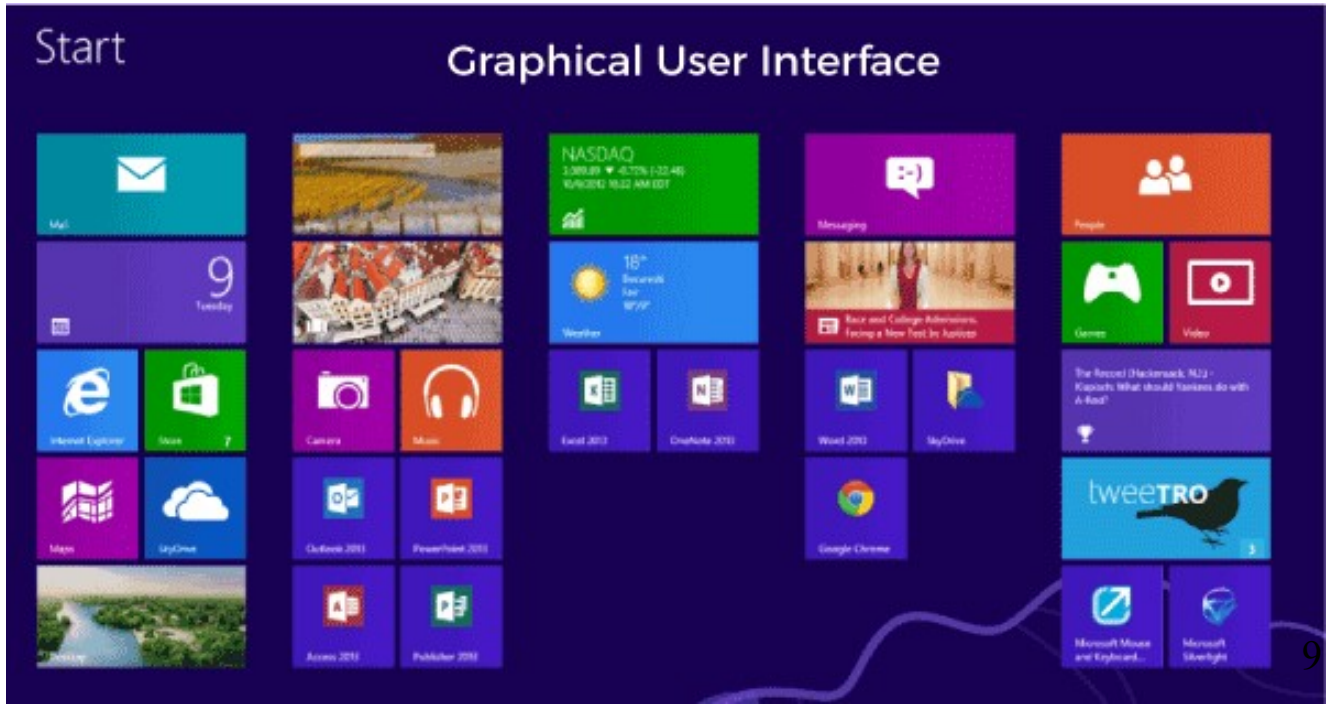
•A tab is associated with a view pane. It usually contains a text label or a graphical icon. Tabs are sometimes related to widgets and multiple tabs allow users to switch between different widgets. Tabs are used in various web browsers such as Internet Explorer, Firefox, Opera, Safari etc. Multiple web pages can be opened in a web browser and users can switch between them using tabs.

GUI Operating System

- An operating system is software that provides an interface between the user and the hardware. The computer cannot perform any task without an operating system. The operating system controls and manages the operation of computer hardware.
- The evolution of operating systems had made PCs easier to use and understand, flexible and reliable. Some popular, modern graphical user interfaces include Microsoft Windows, macOS, Ubuntu Unity, and GNOME Shell for desktop environments, Android, Apple's iOS, BlackBerry OS, Windows 10 Mobile, Palm OS-WebOS, and Firefox OS for smartphones.
- Modern operating systems and graphical user interfaces are incorporated into every interactive application, such as ATMs, video games, smartphones, self-service checkouts, airline self-ticketing and check-in, and desktops.
- Some operating systems are graphical user interfaces (GUI), and some are command-line interfaces (CLI).

What is GUI OS?

•GUI stands for Graphical User Interface. It is a visual representation of communication presented to the user for easy interaction with the machine. The actions in a GUI are usually performed through direct manipulation of graphical elements like buttons and icons. Communication can be performed by interacting with these icons rather than the usual text-based or command-based communication.



What is GUI OS?

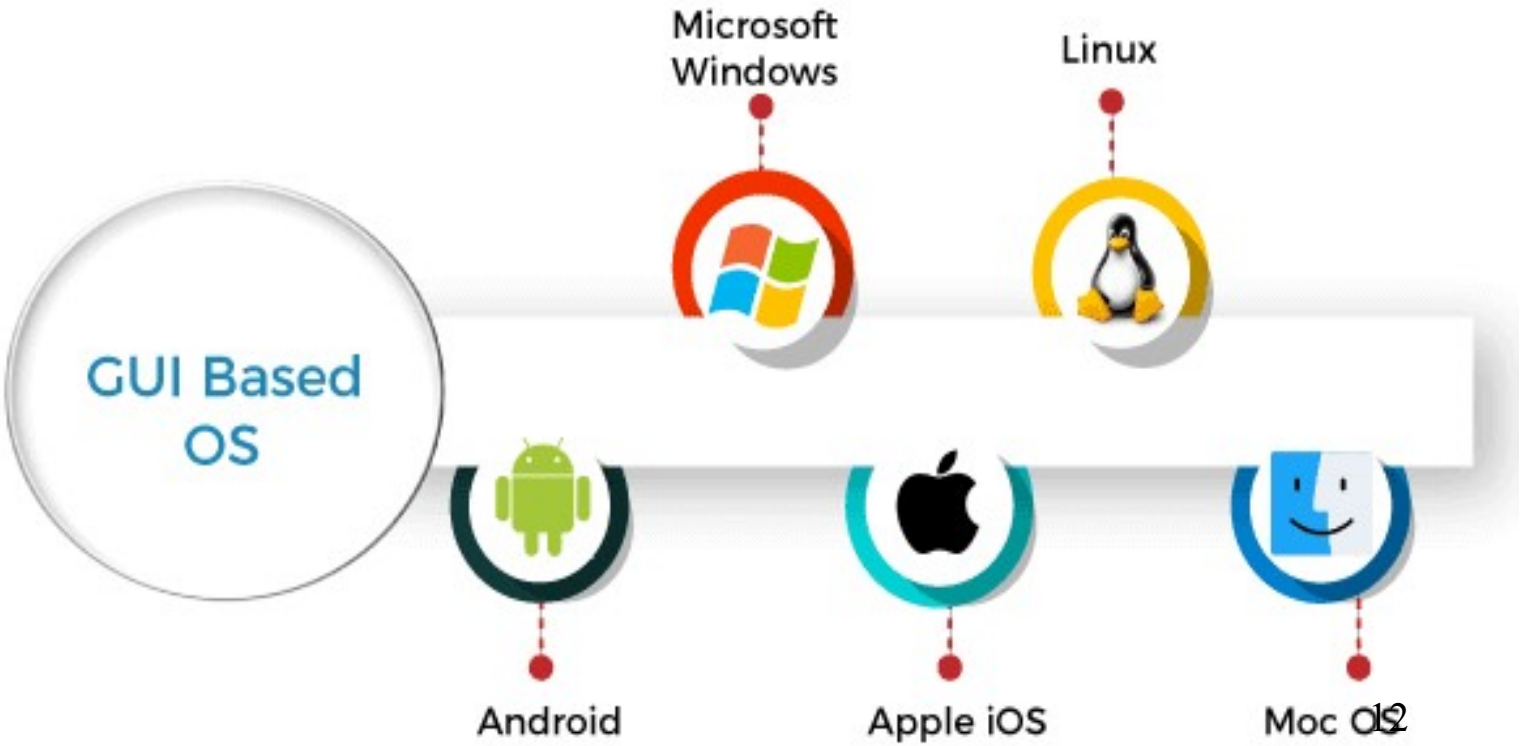
- There are several different visual programming languages with their unique advantages for developing a graphical user interface design. C # or Java may be preferable due to their ability to run GUIs simultaneously in a browser and as a desktop application. Other languages are also be prefered, such as Python, HTML5/Javascript, and C/C++.
- There are different elements and objects that the user use to interact with the software to make a user friendly GUI.
- **Button:** A graphical representation of a button that acts as a program when pressed.
- **Dialogue box:** A type of window that displays additional information and asks a user for input.
- **Icon:** Small graphical representation of a program, features, or file.

What is GUI OS?

- **Menu:** List of commands or choices offered to the user through the menu bar.
- **Ribbon:** Replacement for the file menu and toolbar that groups programs activities together.
- **Tab:** Clickable area at the top of a window that shows another page or area.
- **Toolbar:** Row of buttons, often near the top of an application window, that controls software functions.
- **Window:** Rectangular section of the computer's display that shows the program currently being used.

Types of GUI based Operating System

•***Sketchpad***, believed to be the first graphical computer-aided design program, was developed in 1962 by ***Ivan Sutherland*** at MIT and consisted of a light pen that enabled users to create and manipulate objects in engineering drawings in real time with coordinated graphics. The different types of the operating system based on GUI are as follows:



Types of GUI based Operating System

1. Microsoft Windows

•Microsoft Windows is one of the most common GUI based operating systems. It is developed and marketed by Microsoft. The current version of Microsoft Windows is Windows 11. Moreover, it has several earlier versions like Windows XP, Windows 10, Windows 8, Windows 7, etc. Besides, the most common editions for home computers are Windows Home (Win Home) and Windows Professional (Win Pro or Windows Pro).

2. Linux

•Linux is a popular operating system. It is not a part of any particular firm or organization. Rather, it was developed by a programmer ***Linus Torvalds*** in 1991. It is an open-source operating system. Today, LINUX is run in many organizations, private offices, on mobiles, supercomputers, over the internet, etc.

Types of GUI based Operating System

•It already contains almost all the features of UNIX OS. Moreover, it also has some additional features. It contains various user interfaces like ***GNOME, KDE, Mate, Cinnamon***, etc. And it has several versions of distributions as follows ***UBUNTU, DEBIAN, SOLAS, LINUX MINT***, etc.

3. Android

•It is one of the most common versions of the operating system today. It is used on mobile phones and tablets. Most versions of it are the open-source operating system.

4. Apple iOS

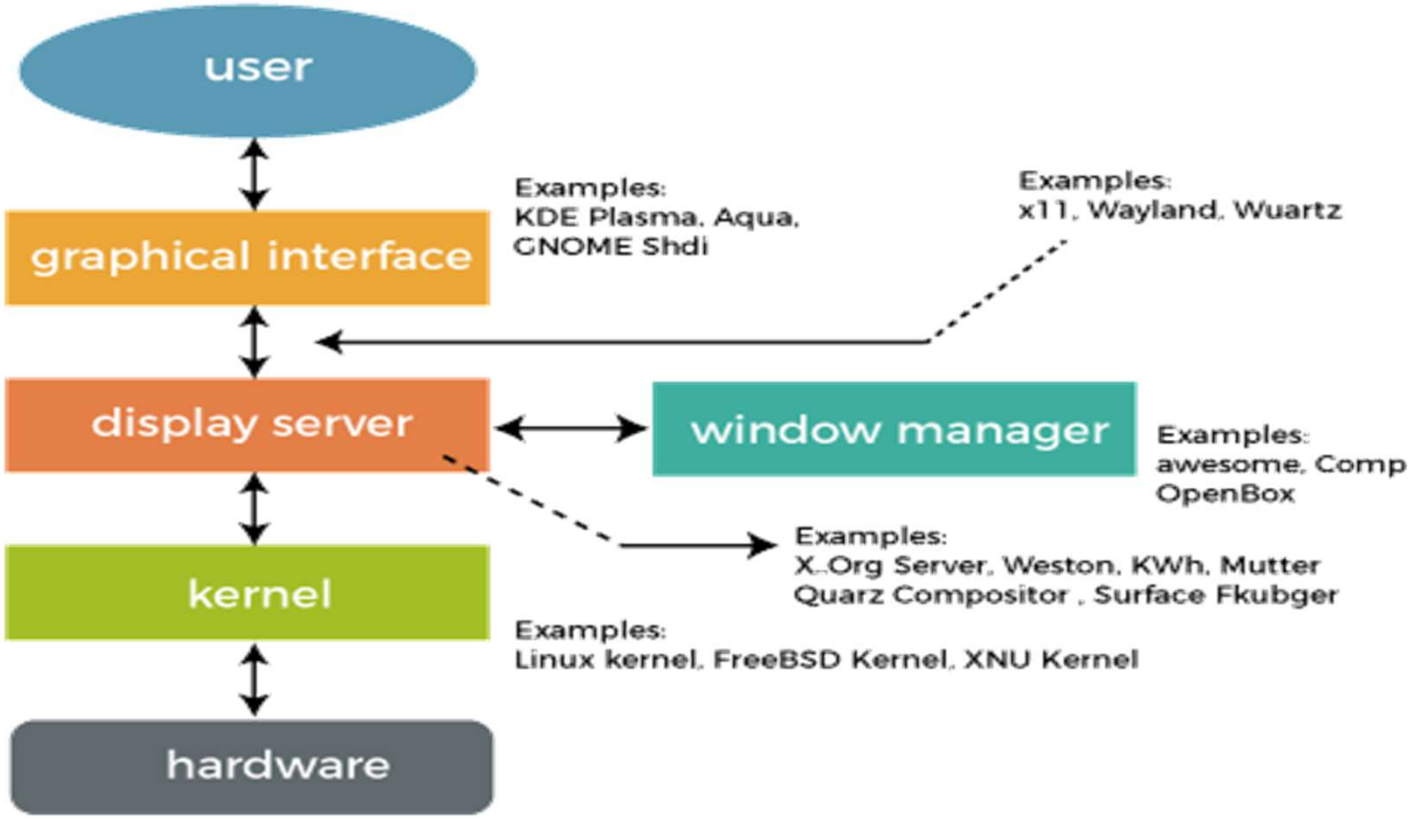
•After android, it is one of the most popular OS. It is designed to run on Apple devices such as iPhones, iPad tablets, etc.

•5. MacOS

•It is the operating system used in Apple laptops and Apple desktops. Moreover, Its most popular features are ***Siri*** and ***FaceTime***.

How Graphical User Interface Works?

•Graphical user interface design principles follow the model-view-controller software pattern, which separates internal representations of information from how information is presented to the user, resulting in a platform where users are shown which functions are possible rather than requiring the input of command codes.



How Graphical User Interface Works?

- Users interact with information by manipulating visual widgets, designed to respond by the type of data they hold and support the actions necessary to complete the user's task.
- The appearance of an operating system or application software may be redesign due to the nature of graphical user interfaces independent of application functions.
- Applications typically implement their own unique graphical user interface display elements and already present on the existing operating system.
- A typical graphical user interface also includes standard formats for representing graphics and text, sharing data between applications running under common graphical user interface design software.
- Graphical user interface testing refers to the systematic process of generating test cases to evaluate the system's functionality and design elements.
- Graphical user interface testing tools (manual or automated) are available under various licenses and supported by various platforms.

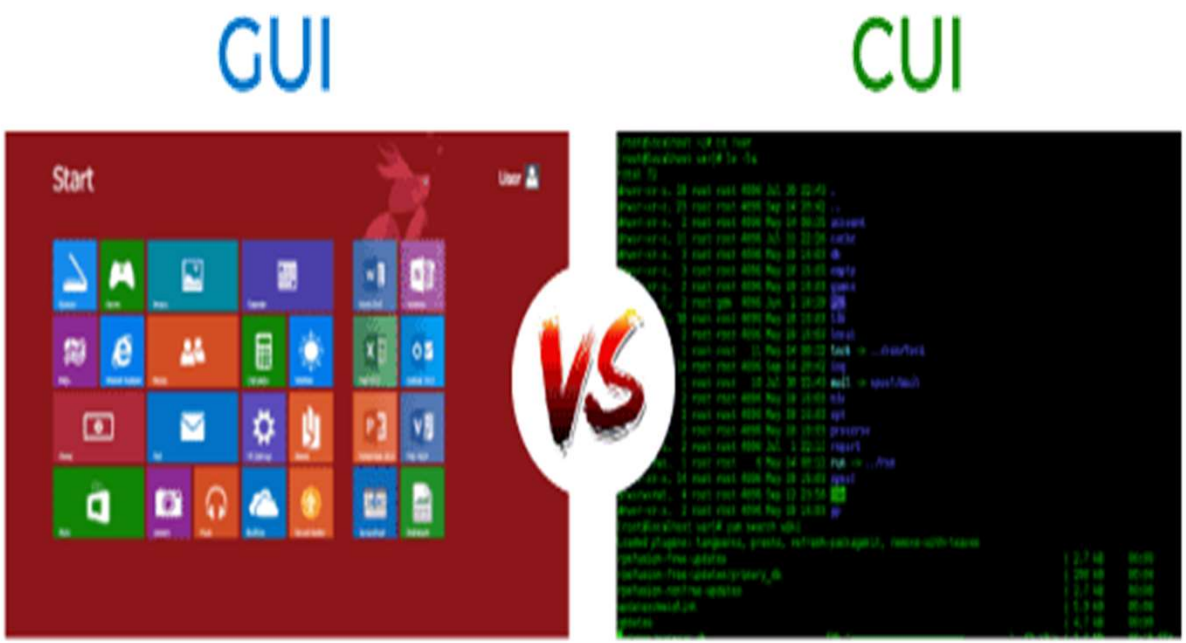
Advantages of Graphical User Interfaces

•Graphical user interfaces operating system has the following most common advantages:

- The advantage of a graphical user interface is a stark improvement in usability for the average user.
- The features of a graphical user interface familiar leverage metaphors, such as drag-and-drop for transferring files, and use familiar icons, such as a trash bin for deleted files, creating an environment in which computer operations are intuitive and easy to use without any prior practise or knowledge of computing machinery or languages.
- Graphical user interface applications are self-descriptive, feedback is typically immediate, and visual cues encourage and steer discoverability.

Difference between GUI and CUI Operating System

•The interface can be understood as the medium we communicate with each other. In the same way, to communicate with the computer, we have interfaces. There are two types of interface one is the graphical user interface, and the other one is the command-line interface.



Difference between GUI and CUI Operating System

- A command user interface (CUI) processes commands to a computer program in the form of lines of text. The program which handles the interface is called a command-line interpreter or command-line processor. Operating systems implement a command-line interface in a shell for interactive access to operating system functions or services.
- The graphical user interface (GUI) allows users to interact with electronic devices through graphical icons and audio indicators such as primary notation instead of text-based user interfaces, typed command labels, or text navigation. Below are some more differences between GUI and CUI Operating System, such as:

Difference between GUI and CUI Operating System

Terms	GUI	CUI
Interaction	A user interacts with the computer using Graphics like images, icons.	A user interacts with a computer using commands like text.
Navigation	Navigation is easy.	Navigation is difficult.
Precision	GUI has low precision.	CUI has high precision.
Usage	GUI is easy to use.	CUI is difficult to use and requires expertise.
Speed	GUI is of low speed.	CUI is of high speed.
Peripherals used	GUI operating system requires a keyboard, mouse or any other pointing device.	CUI operating system requires the only keyboard.
Memory requirement	It required high memory.	It required low memory.
Flexibility	Highly flexible user interface.	Less flexible user interface.
Customize	GUI is highly customizable.	CUI appearance is not easily changeable.

Java Foundation Classes (JFC)

JFC Java

- In 1997, Sun Microsystems and IBM decided to solve the problem of access-enabling software.
- Their aim was to develop such an accessibility API that application developers could implement on Java class libraries to make applications accessible.
- As the result, Sun Microsystems wrote the accessibility API and IBM has tested their API and implemented it on their foundation classes.
- Sun Microsystems integrated JFC/ Swing directly into Java 1.2 and later versions.
- In this, we will discuss the **JFC, its components, services,** and **the difference between JFC** and **AWT**.
-

What is JFC?

- JFC stands for **Java Foundation Classes**.
- It is a rich and comprehensive set of GUI components and services that simplify the development and deployment of desktop, client-side, and internet applications.
- It is a superset that contains AWT. JFC extends AWT by adding many components and services that are as follows:
  - ❑ Swing GUI Components
  - ❑ Pluggable Look-and-Feel Support
  - ❑ Accessibility API
  - ❑ Internationalization
  - ❑ Drag and Drop (DnD)

What is JFC?

- JFC stands for **Java Foundation Classes**.
- It is a rich and comprehensive set of GUI components and services that simplify the development and deployment of desktop, client-side, and internet applications.
- It is a superset that contains AWT. JFC extends AWT by adding many components and services that are as follows:

API/ Feature	Description
Swing GUI Components	It includes everything from buttons to split panes to tables. Many components are capable of sorting, printing, and drag and drop, to name a few of the supported features.

What is JFC?

API/ Feature	Description
Pluggable Look-and-Feel Support	The look and feel of Swing applications is pluggable, allowing a choice of look and feel. For example, the same program can use either the Java or the Windows look and feel. Additionally, the Java platform supports the GTK + look and feel, which makes hundreds of existing looks and feels available to Swing programs. Many more look-and-feel packages are available from various sources.
Accessibility API	It is a part of JFC that works with alternate input and output devices. It enables assistive technologies, such as screen readers, screen magnifiers, and Braille terminals, to get information from the UI.

What is JFC?

API/ Feature	Description
Java 2D API	It enables developers to easily incorporate high-quality 2D graphics, text, and images in applications and applets. Java 2D includes extensive APIs for generating and sending high-quality output to printing devices.
Internationalization	It allows developers to build applications that can interact with users worldwide in their own languages and cultural conventions. With the input method framework, developers can build applications that accept text in languages that use thousands of different characters, such as Japanese, Chinese, or Korean.
Drag and Drop (DnD)	Drag and Drop is one of the more common metaphors used in GUI. The user is allowed to click and "hold" a GUI object, moving it to another window or frame in the desktop with predictable results. It allows users to implement droppable elements that transfer information between Java applications and native applications. Although DnD is not part of Swing, it is crucial to a commercial-quality application.

What is JFC?

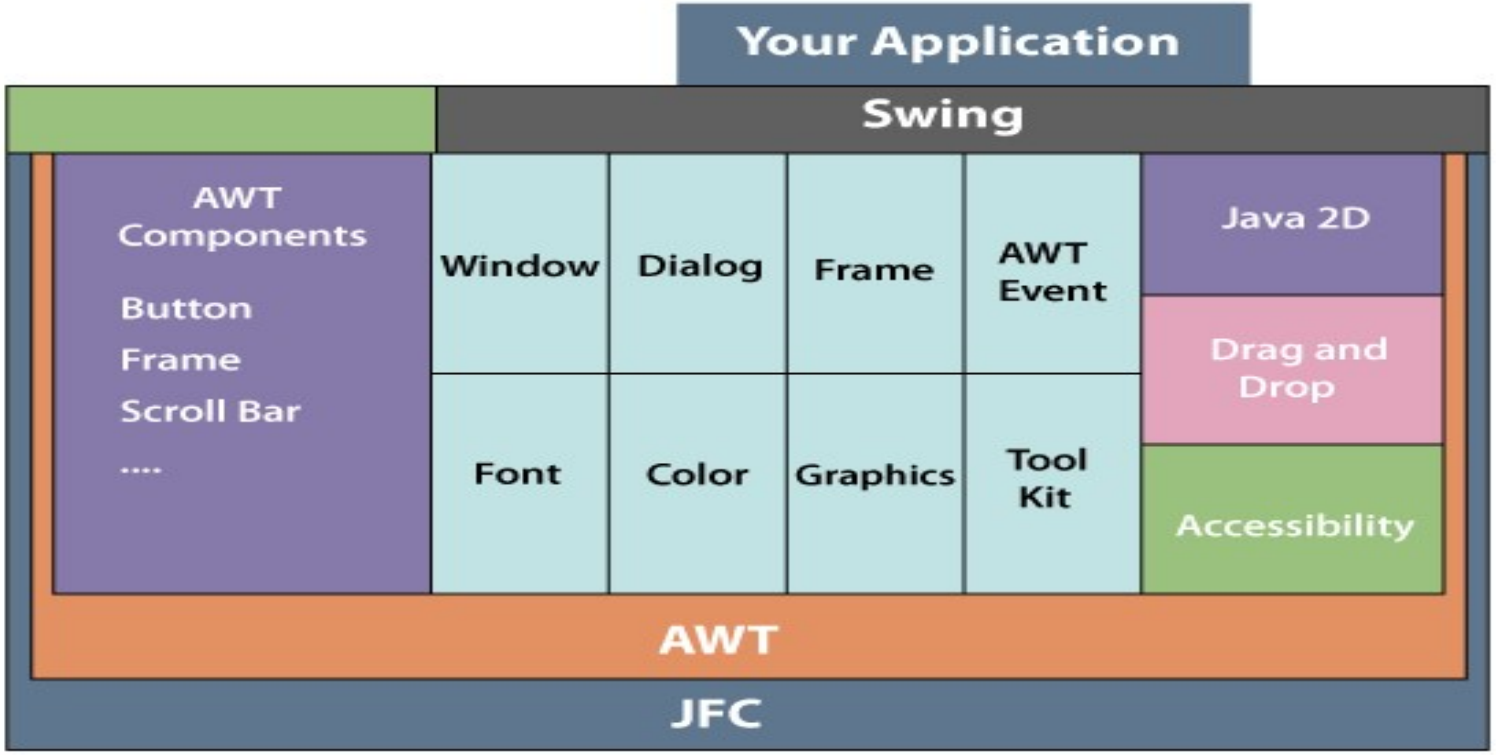
- Sun Microsystems added a mechanism to the JFC is called Multiplexed UI. It is an extension to the PLAF (Pluggable Look & Feel). A key feature of the PLAF is that it enables developers to swap the Look and Feel of the UI which can be directly accessible. It was developed for application developers to provide a common interface.

The Multiplexed UI mechanism allows the developer to:

- Add multiple alternative UI to the JFC component library while JVM starts.
- Makes multi-modal access (such as hands-free voice access) features possible with a smaller amount of development effort.

JFC architecture

- The following figure describes the JFC architecture.



- In the above figure, see that Swing sits atop of part, but not all, of the AWT components. Swing sits on a number of the APIs that implement the various parts of AWT, including Java2D, Drag-and-Drop, and the Accessibility API.

Advantage of JFC

- Its components are pluggable and require few lines of code.
- It retains Java qualities.
- An application that runs flawlessly on one OS runs flawlessly on another OS.
- It offers an open architecture.
- Accordingly, the exhibition of a GUI made through JFC is unsurprising. Therefore, JFC are widely used in client-side applications.

Components of JFC

- The JFC components includes all the JavaBeans and swing Components that offer re-usability, interoperability, and portability.
- Note that Swing is an enhancement of AWT, not a replacement.

Difference Between JFC and WFC

JFC	WFC
It supports robust and portable interfaces.	It runs only on the Windows (32-bit) user interface that require Microsoft extension.
It provides greater control over user interface because of object-oriented nature.	The number of controls available for WFC is probably smaller than for JFC.
It requires more memory.	It requires less memory.
Its performance is slower than WFC.	Its performance is faster than JFC.
It is used to build complex user interface.	It is used to build simple user interface.

Java AWT (Abstract Window Toolkit)

- **AWT (Abstract Window Toolkit)** is a part of the Java Foundation Classes (JFC) used to create GUI (Graphical User Interface) or window-based applications. It provides platform-independent libraries but relies on native OS components for rendering, making it a heavyweight and platform-dependent application.
  - ❑ Part of **java.awt** package.
  - ❑ Provides GUI components like **Button, Label, TextField, Checkbox, Choice, List, Canvas**, etc.
  - ❑ Heavyweight components: Depend on the underlying OS for look and feel.
  - ❑ Platform-dependent appearance: AWT apps look like Windows apps on Windows, Mac apps on macOS, etc.

Java AWT (Abstract Window Toolkit)

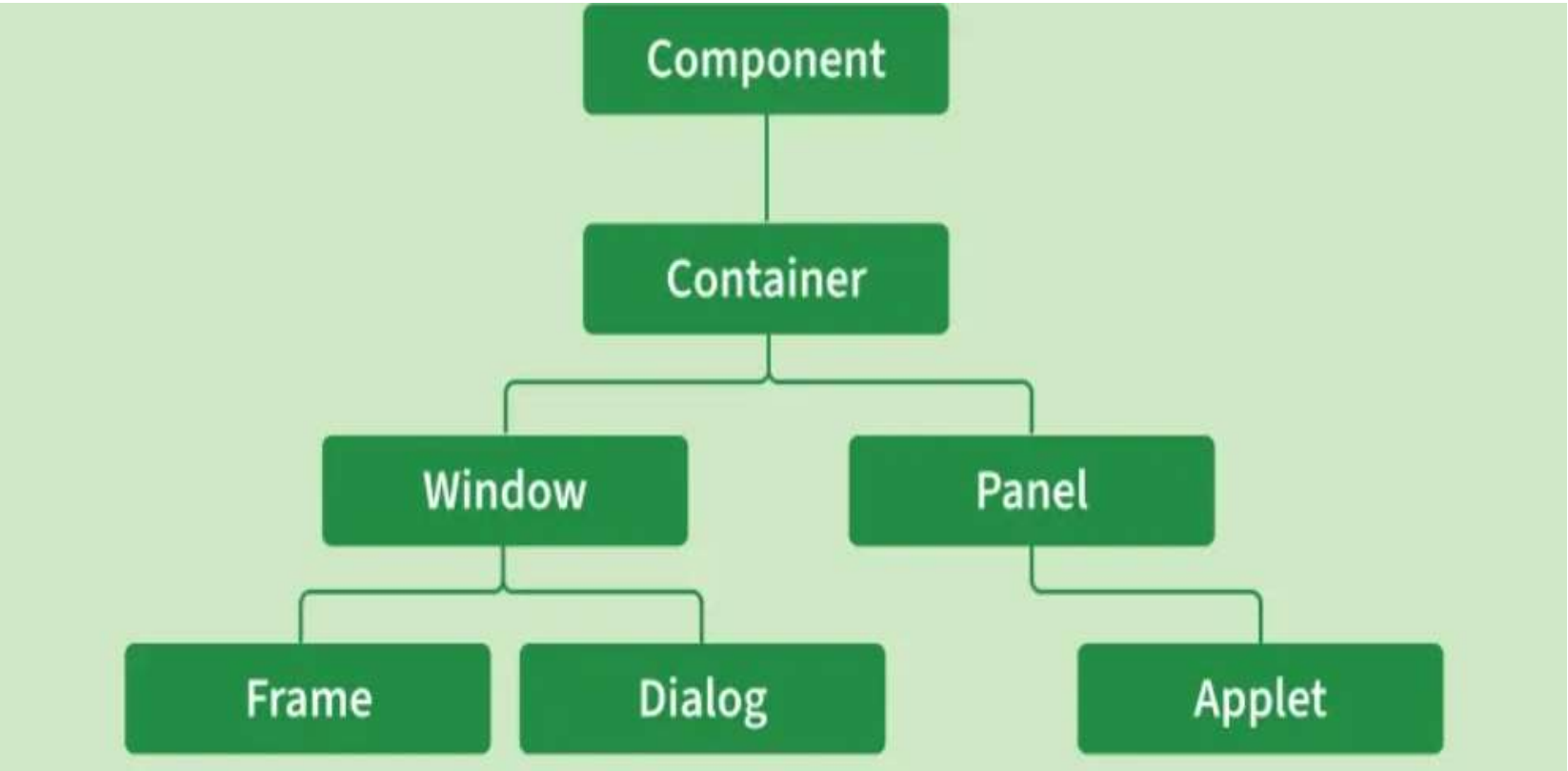
Types of Containers in Java AWT

There are four types of containers in Java AWT:

- **Window:** Window is a top-level container that represents a graphical window or dialog box. The Window class extends the Container class, which means it can contain other components, such as buttons, labels and text fields.
- **Panel:** Panel is a container class in Java. It is a lightweight container that can be used for grouping other components within a window or a frame.
- **Frame**: The Frame is the container that contains the title bar and border and can have menu bars.
- **Dialog:** A dialog box is a temporary window an application creates to retrieve user input.

Java AWT (Abstract Window Toolkit)

AWT Hierarchy



Java AWT (Abstract Window Toolkit)

Common AWT Components & Constructors

- Label
- Button
- TextField
- Checkbox
- CheckboxGroup
- Choice
- List
- Canvas
- Scrollbar
- MenuItem & Menu
- PopupMenu
- Panel
- Toolkit

Java AWT (Abstract Window Toolkit)

Java AWT Label

- Label in Java AWT is a component for placing text or images in a container. The text displayed in the Container (Display) by Java AWT Label can't be edited directly by the user but needs the programmer to change it from his side.
- *Java AWT Label is also called **passive control** as no event is created when it is accessed.*
- **Syntax of Java AWT Label**

public class Label **extends** Component **implements** Accessible

Java AWT (Abstract Window Toolkit)

Java AWT Label

- **Java AWT Label Class Constructors**

There are three types of Java AWT Label Class

1. Label():

Creates Empty Label.

2. Label(String str):

Constructs a Label with str as its name.

3. Label(String str, int x):

Constructs a label with the specified string and x as the specified alignment

Java AWT (Abstract Window Toolkit)

Java AWT Label Class Methods

Sr. No.	Method	Description
1.	**void setText(String text)**	Sets the text of the label.
2.	**String getText()**	Gets the text of the label.
3.	**void setAlignment(int alignment)**	Sets the alignment of the text in the label. Possible values: Label.LEFT, Label.RIGHT, Label.CENTER.
4.	**int getAlignment()**	Gets the alignment of the text in the label.
5.	**void addNotify()**	Creates the peer for the label.
6.	**AccessibleContext getAccessibleContext()**	Gets the AccessibleContext associated with the label.
7.	**void addNotify()**	Creates the peer (native window) for the label.

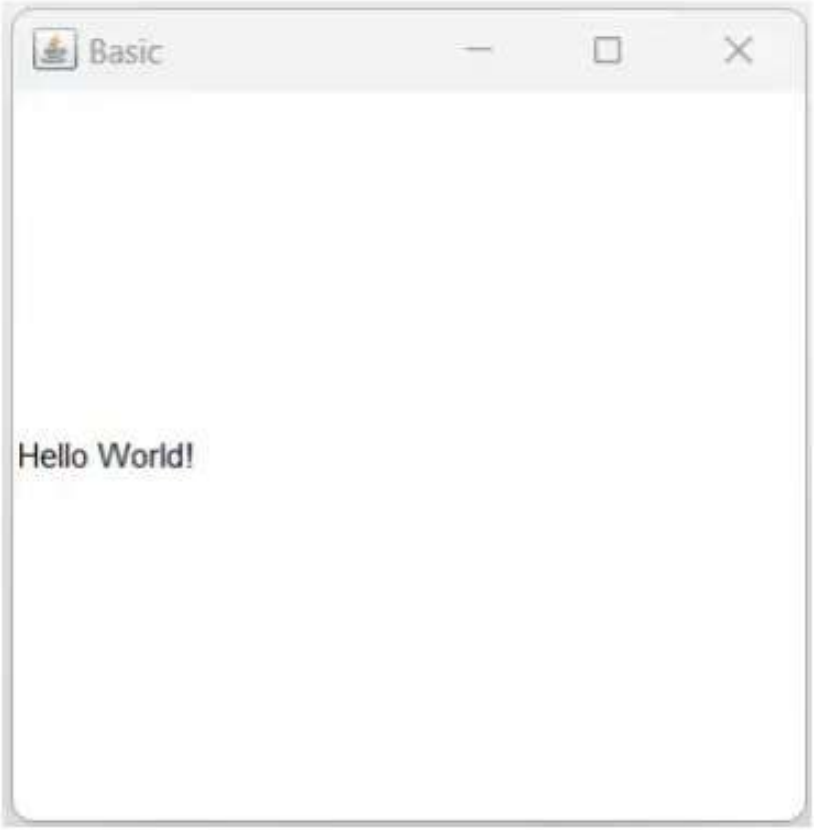
Java AWT (Abstract Window Toolkit)

Java AWT Label

```
// Java AWT Program to demonstrate Use of Label
// Hello World Program in AWT
import java.awt.*;
import java.awt.event.WindowAdapter;
import java.awt.event.WindowEvent;

// Driver Class
class Main {
    // Main Function
    public static void main(String[] args)
    {
        // Declaring Frame
        Frame frame = new Frame("Basic");
        // All the components will be
        // inserted in this frame

        // Declaring Label
        Label label = new Label("Hello World!");
        // Setting the Aligment to CENTER
        // label.setAlignment(Label.CENTER);
        // Adding Label into Frame
        frame.add(label);

        // Setting Frame Size
        frame.setSize(500, 500);
        // Setting Visibility of Frame
        frame.setVisible(true);

        // Using WindowListener for closing the window
        frame.addWindowListener(new WindowAdapter() {
            @Override
            public void windowClosing(WindowEvent e)
            {
                System.exit(0);
            }
        });
    }
}
```

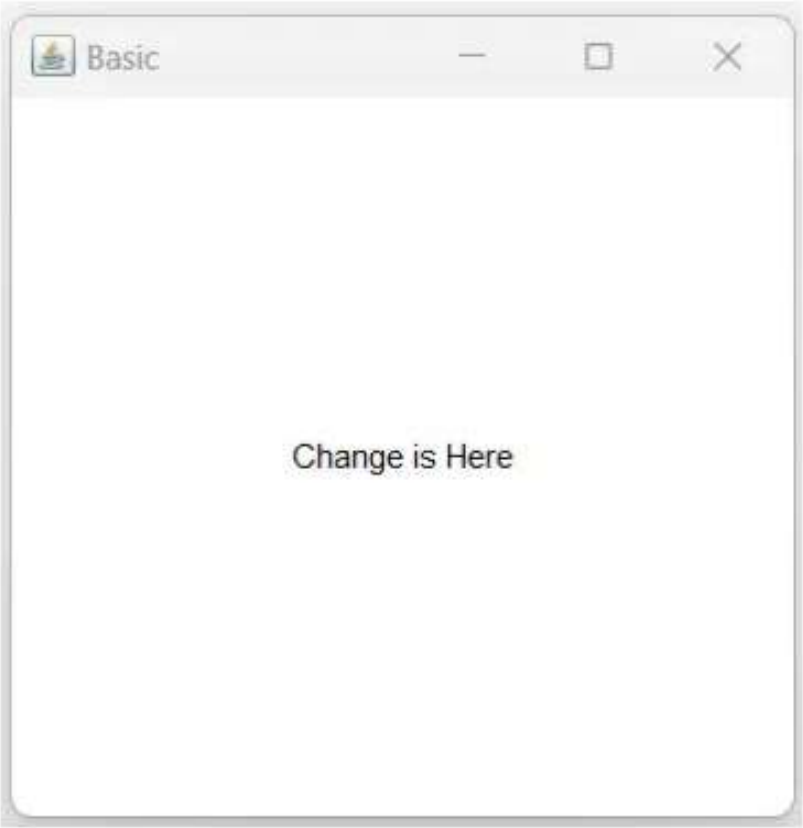
Java AWT (Abstract Window Toolkit)

Java AWT Label



Java AWT (Abstract Window Toolkit)

Java AWT Label



Java AWT (Abstract Window Toolkit)

Java AWT Button

- The **'Button'** class is a part of the 'java.awt' package which has a collection of classes and methods for creating GUI components.
- Java AWT Buttons can be used to perform several actions like saving a file, closing a window, submitting a form, or triggering any specific action.
- When we press a button, AWT creates an instance of **ActionEvent** and delivers it by calling **processEvent** on the button.
- The **processEvent** method of the button accepts all events and then sends an action event to its own function **processActionEvent**.
- The **ActionListener** interface must be implemented if one wants to execute an action when a button is pressed.

Java AWT (Abstract Window Toolkit)

- **Syntax of Java AWT Button**

public class Button **extends** Component **implements** Accessible

Constructors of Button

There are 2 types of Constructors in the AWT Button class.

- **Button()**: Creates a new button with the label as an empty string.
- **Button(String label)**: Creates a new button with the label as the specified string as a parameter.

Java AWT (Abstract Window Toolkit)

Java AWT Button Class methods

Method	Description
void addActionListener(ActionListener l)	The action listener is added to receive action events from this button.
void addNotify()	Creates the **peer** of the button.
AccessibleContext getAccessibleContext()	The method returns the **AccessibleContext** associated with this Button.
String getActionCommand()	The method returns the command name of the action event triggered by this button.
ActionListener[] getActionListeners()	The method returns an array of all the action listeners that have been registered on this button.
String getLabel()	Retrieves the label of this button.
<T extends EventListener> T[] getListeners(Class<T> listenerType)	The method returns an array of all the objects that are currently registered as FooListeners on this Button.

Java AWT (Abstract Window Toolkit)

Java AWT Button Class methods

Method	Description
protected String paramString()	Returns a string that represents the status of this Button.
protected void processActionEvent(ActionEvent e)	Sends action events to any registered ActionListener objects in order to process them when they occur on this button.
protected void processEvent(AWTEvent e)	Processes events on this button.
void removeActionListener(ActionListener l)	The specified action listener is removed from the button.
void setActionCommand(String command)	Configures the action event by setting the command name.
void setLabel(String label)	Sets the button's label to be the specified string.

Java AWT (Abstract Window Toolkit)

Java AWT Button

```
// Java AWT Program to create Basic Button
import java.awt.Button;
import java.awt.Frame;

// Driver Class
public class Main {
     // main function
   public static void main(String[] args)
   {
      // Create a frame
      Frame frame = new Frame("AWT Button
Example");
      // Create a button
      Button button = new Button("Click Me!");
// Set the button position on the frame
      button.setBounds(150, 130, 100, 30);

      // Add the button to the frame
      frame.add(button);

      // Set the frame size and layout
      frame.setSize(400, 300);
      frame.setLayout(null);

      // Set the frame visibility to true
      frame.setVisible(true);
   }
}
```

Java AWT (Abstract Window Toolkit)

Java AWT Button

```java
import java.awt.Button;
import java.awt.Frame;

public class Main {
    Run | Debug
    public static void main(String[] args)
    {

        // Create a frame
        Frame frame = new Frame(title:"AWT Button Example");
        // Create a button
        Button button = new Button(label:"Click Me!");

        // Set the button position on the frame
        button.setBounds(x:150, y:130, width:100, height:30);

        // Add the button to the frame
        frame.add(button);

        // Set the frame size and layout
```

PROBLEMS OUTPUT TERMINAL PORTS DEBUG CONSOLE

```
PS E:\Web Dev Projects\GFG\Java\AWT>
```

Java AWT (Abstract Window Toolkit)

Java AWT Button (Adding Multiple)

```
// Java AWT Program for
// Adding Multiple Properties
import java.awt.*;
import java.awt.Font;

// Driver Class
public class Main {
    // main function
  public static void main(String[] args){

      // Create a frame
      Frame frame = new Frame("AWT Button Example");

      // Button 1
       Button button1 = new Button("Click Me!");

      // Set the background color
      button1.setBackground(Color.BLUE);
      // Set the foreground color
      button1.setForeground(Color.WHITE);
      // Set the button font size
      button1.setFont(new Font("Arial", Font.BOLD, 14));
      // Set the button position
      button1.setBounds(150, 50, 100, 50);

      // Button 2 with different properties
       Button button2 = new Button("Press Me!");
      button2.setBackground(Color.GREEN);
      button2.setForeground(Color.BLACK);
button2.setFont(new Font("Times New Roman", Font.PLAIN, 16));
      button2.setBounds(150, 140, 80, 30);

      // Button 3 with different properties
      Button button3 = new Button("Tap Me!");
      button3.setBackground(Color.RED);
      button3.setForeground(Color.WHITE);

      button3.setFont(new Font("Verdana", Font.ITALIC, 16));
      button3.setBounds(130, 220, 150, 80);

      // Add the buttons to the frame
      frame.add(button1);
      frame.add(button2);
      frame.add(button3);

      // Set the frame background color
      frame.setBackground(Color.LIGHT_GRAY);

      // Set the frame size and make it visible
      frame.setSize(400, 400);
      frame.setLayout(null);

      // Set the frame visibility to true
      frame.setVisible(true);
  }
}
```

Java AWT (Abstract Window Toolkit)

Java AWT Button (Adding Multiple)

```
AWT
Main.java

Main.java > Main > main(String[])
import java.awt.*;
import java.awt.Font;

public class Main {
    Run | Debug
    public static void main(String[] args)
    {

        // Create a frame
        Frame frame = new Frame(title:"AWT Button Example");

        // Button 1
        Button button1 = new Button(label:"Click Me!");

        // Set the background color
        button1.setBackground(Color.BLUE);

        // Set the foreground color
        button1.setForeground(Color.WHITE);

PROBLEMS   OUTPUT   TERMINAL   PORTS   DEBUG CONSOLE

PS E:\Web Dev Projects\GFG\Java\AWT> j

> OUTLINE
```

Java AWT (Abstract Window Toolkit)

Java AWT TextField

- The **TextField** class in Java AWT is a GUI component in the ***'java.awt'*** package. It allows the user to enter a single line of text as an input.
- It is a simple way to collect text-based information from the user. You can use the **TextField** component to create various input fields for username, password, search boxes, etc.
- Whenever a key is pressed or typed into the input field, an event is sent to the **TextField.** The registered **KeyListener** then receives the **KeyEvent**.
- **ActionEvent** can also be used for this purpose. If the ActionEvent is configured on the text field, then it will be triggered when the **Return** key is pressed. The **ActionListener** interface handles the event.

Java AWT (Abstract Window Toolkit)

Java AWT TextField

- **Syntax of Java AWT TextField**

public class **TextField** extends **TextComponent**

Class Constructors of AWT TextField

- **TextField()**: Constructs a new text field component
- **TextField(int columns)**: Constructs a new empty text field with the required number of columns as the parameter.
- **TextField(String text)**: Creates a new text field initialized with the string text to be displayed.
- **TextField(String text, int columns)**: Creates a new text field with the given text to be displayed and a width that fits the specified number of columns.

Java AWT (Abstract Window Toolkit)

Class Methods of AWT TextField

Method	Description
void addActionListener(ActionListener l)	This action listener is added to receive action events from this text field.
void addNotify()	Creates a peer for the TextField.
boolean echoCharIsSet()	Check if there is a character set for echoing in this text field.
AccessibleContext getAccessibleContext()	Retrieves the AccessibleContext associated with the TextField.
ActionListener[] getActionListeners()	An array of all action listeners registered on a textfield is returned.
int getColumns()	Returns the number of columns in a text field.
char getEchoChar()	Returns the character to be used for echoing.
Dimension getMinimumSize()	It returns the minimum dimensions for a text field.
Dimension getMinimumSize(int columns)	Fetches the minimum dimensions for a text field with the specified number of columns.

Java AWT (Abstract Window Toolkit)

Class Methods of AWT TextField

Method	Description
Dimension getPreferredSize()	Returns the preferred size of a text field.
Dimension getPreferredSize(int columns)	It returns the preferred size of a text field with specified number of columns.
protected String paramString()	Returns a string indicating the state of this TextField.
protected void processActionEvent(ActionEvent e)	Action events that occur on the text field are processed by dispatching them to any registered ActionListener objects.
protected void processEvent(AWTEvent e)	Processes an event on the text field.
void removeActionListener(ActionListener l)	It removes the specified action listener from the text field..
void setColumns(int columns)	Defines the number of columns in the text field.
void setEchoChar(char c)	Defines echo character for the text field.
void setText(String t)	Sets the text that is presented by this text component to be the specified text.

Java AWT (Abstract Window Toolkit)

Class Methods of AWT TextField

```
// Java AWT Program to demonstrate
// Use of TextField
import java.awt.*;

// Driver Class
public class Main {
    // main function
   public static void main(String[] args){

      // Create a frame
      Frame f = new Frame("TextField Example");

      // Create a text field
      TextField t1 = new TextField();

       // Set position
      t1.setBounds(50, 50, 200, 30);
      TextField t2 = new TextField();
      t2.setBounds(50, 100, 200, 30);
```

```
// Add text fields to the frame
      f.add(t1);
      f.add(t2);

      // Set frame size
      f.setSize(300, 200);

      // Set frame layout
      f.setLayout(null);

      // Make the frame visible
      f.setVisible(true);
   }
}
```

Java AWT (Abstract Window Toolkit)

Java AWT Checkbox

- Java AWT (Abstract Window Toolkit) provides a really various set of tools for edifice in writing user interfaces (GUIs), and among these tools is the **Checkbox class**.
- **Checkboxes** are necessary components for user interactions, allowing users to work binary choices easily.

Syntax of Java AWT Checkbox

public class Checkbox extends Component implements ItemSelectable, Accessible

Java AWT (Abstract Window Toolkit)

Constructors of Checkbox class

Constructors	Description
Checkbox()	Creates a checkbox with no label.
Checkbox(String label)	Creates a checkbox with the specified label.
Checkbox(String label, boolean state	Creates a checkbox with the given label and sets the initial state.
Checkbox(String label, boolean state, CheckboxGroup group)	Creates a checkbox with the specified label, state, and associates it with a CheckboxGroup.
Checkbox(String label, CheckboxGroup group, boolean state)	Creates a checkbox with the given label, associates it with the specified CheckboxGroup, and sets the initial state.

Java AWT (Abstract Window Toolkit)

Key Methods of Checkbox class

Method	Description
void addItemListener(ItemListener IL)	Adds the specified item listener to receive events when the state of the checkbox changes.
AccessibleContext getAccessibleContext()	Retrieves the accessible context for enhanced interaction with assistive technologies for the checkbox.
void addNotify()	Creates the peer of the checkbox for rendering purposes.
CheckboxGroup getCheckboxGroup()	Retrieves the associated CheckboxGroup to which the checkbox belongs.
ItemListener[] getItemListeners()	Returns an array of registered item listeners responsible for handling item events of the checkbox.
String getLabel()	Retrieves the label associated with the checkbox.
T[] getListeners(Class listenerType)	Returns an array containing the label of the checkbox if it is selected; otherwise, returns null.
boolean getState()	Returns true if the checkbox is in the selected state; otherwise, returns false.

Java AWT (Abstract Window Toolkit)

Key Methods of Checkbox class

Method	Description
protected String paramString()	Generates a string representation of the state and properties of the checkbox.
protected void processEvent(AWTEvent e)	Processes the general events occurring on the checkbox.
protected void processItemEvent(ItemEvent e)	Processes the item events occurring in the checkbox by dispatching them to registered ItemListener objects.
void removeItemListener(ItemListener l)	Removes the specified item listener to stop receiving item events from the checkbox.
void setCheckboxGroup(CheckboxGroup g)	Sets the checkbox's group to the specified CheckboxGroup.
void setLabel(String label)	Sets the label of the checkbox to the specified string argument.
void setState(boolean state)	Sets the state of the checkbox to the specified state, either selected (true) or deselected (false).

Layout Managers

- Layout managers control component arrangement in containers.

  - ❑ **FlowLayout**
  - ❑ **BorderLayout**
  - ❑ **GridLayout**
  - ❑ **CardLayout**

Layout Managers (FlowLayout)

- FlowLayout is used to arrange components in a sequence one after the other. The default layout of applet and panel is FlowLayout.

Constructors :

- **FlowLayout()**: It will Construct a new FlowLayout with centered alignment. The horizontal and vertical gap will be 5 pixels.
- **FlowLayout(int align)** : It will Construct a new FlowLayout with given alignment. The horizontal and vertical gap will be 5 pixels.
- **FlowLayout(int align, int HorizontalGap, int VerticalGap)**: It will Construct a new FlowLayout with given alignment, the given horizontal and vertical gap between the components.
- **JLabel(String text)**: It will create a JLabel instance with the specified text.

Layout Managers (FlowLayout)

Commonly used methods:

- **setTitle(String Text)**: This Method is used to set Title of JFrame. The title you want to set is passed as a string.
- **getAlignment()**: Returns the alignment for this layout.
- **setAlignment(int align)**: used to set the alignment for this layout.
- **removeLayoutComponent(Component comp)**: Used to remove the component passed as argument from the layout.

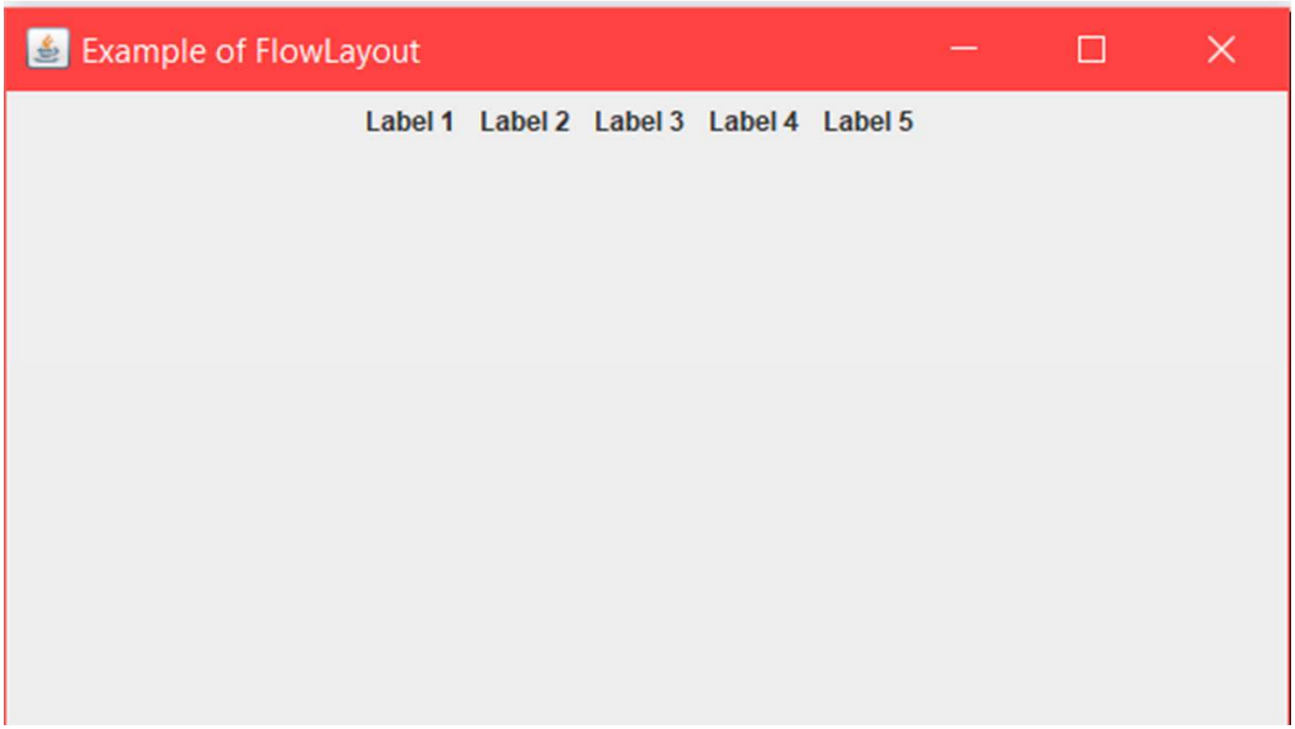
Layout Managers (FlowLayout)

```
// Java program to show Example of FlowLayout.
// in java. Importing different Package.
import java.awt.*;
import java.awt.event.*;
import javax.swing.*;

class Example extends JFrame {
   // Declaration of objects of JLabel class.
   JLabel l1, l2, l3, l4, l5;

   // Constructor of Example class.
   public Example()
   {
      // Creating Object of "FlowLayout" class
      FlowLayout layout = new FlowLayout();

      // this Keyword refers to current object.
      // Function to set Layout of JFrame.
      this.setLayout(layout);

      // Initialization of object "l1" of JLabel class.
      l1 = new JLabel("Label 1  ");
      // Initialization of object "l2" of JLabel class.
      l2 = new JLabel("Label 2  ");
```

```
  // Initialization of object "l3" of JLabel class.
      l3 = new JLabel("Label 3  ");
// Initialization of object "l4" of JLabel class.
      l4 = new JLabel("Label 4  ");

      // Initialization of object "l5" of JLabel class.
      l5 = new JLabel("Label 5  ");

      // this Keyword refers to current object.
      // Adding Jlabel "l1" on JFrame.
      this.add(l1);

      // Adding Jlabel "l2" on JFrame.
      this.add(l2);

      // Adding Jlabel "l3" on JFrame.
      this.add(l3);

      // Adding Jlabel "l4" on JFrame.
      this.add(l4);

      // Adding Jlabel "l5" on JFrame.
      this.add(l5);
   }
}
```

```
class MainFrame {

   // Driver code

   public static void main(String[] args)

   {
      // Creating Object of Example class.
      Example f = new Example();

      // Function to set title of JFrame.

      f.setTitle("Example of FlowLayout");

      // Function to set Bounds of JFrame.

      f.setBounds(200, 100, 600, 400);

      // Function to set visible status of JFrame.
      f.setVisible(true);
   }
}
```

Layout Managers (FlowLayout)



Layout Managers (BorderLayout)

- BorderLayout is the default layout for the window objects such as JFrame, JWindow, JDialog, JInternalFrame etc.
- BorderLayout arranges the components in the five regions. Four sides are referred to as north, south, east, and west. The middle part is called the center.
- Each region can contain only one component and is identified by a corresponding constant as ***NORTH*, *SOUTH*, *EAST*, *WEST*, and *CENTER*.**

Constructors:

- **BorderLayout():** It will construct a new borderlayout with no gaps between the components.
- **BorderLayout(int, int):** It will constructs a border layout with the specified gaps between the components.

Layout Managers (BorderLayout)

Commonly Used Methods:

- **toString()**: Returns a string which is the representation of the state of border layout.
- **getLayoutAlignmentX(Container parent)**: Returns the layout alignment along the X-axis.
- **getLayoutAlignmentY(Container parent)**: It will return the layout alignment along the Y-axis.
- **removeLayoutComponent(Component comp)**: This method is used to remove the specified component from the borderlayout.
- **getVgap()**: Return the vertical gap between the components.
- **getHgap()**: Returns the Horizontal gap between the components.
- **setHgap(int hgap)**: It is used to set the horizontal gap between the components.
- **setVgap(int vgap)**: It is used to set the vertical gap between the components.

Layout Managers (GridLayout)

- GridLayout class represents a layout manager with a specified number of rows and columns in a rectangular grid. The GridLayout container is divided into an equal-sized of rectangles, and one of the components is placed in each rectangle. Every rectangle cell has the same size therefore, they contain a component, which fills the entire cell. When the user changes or adjusts the size of the container, the size of each rectangles changes accordingly.

Constructors of the class:

- **GridLayout():** It Creates a grid layout with a default of one column per component, in a single row.
- **GridLayout(int rw, int cl):** It creates a grid layout with the specified number of rows and columns.
- **GridLayout(int rw, int cl, int hgap, int vgap):** It creates a grid layout with the specified number of rows and columns with horizontal and vertical gap.

Layout Managers (GridLayout)

Commonly Used Methods:

- **addLayoutComponent(String str, Component cmp):** Adds the specified component with the specified name to the layout.
- **setColumns(int cl):** Sets the number of columns in this layout to the specified value.
- **setHgap(int hgap):** Sets the horizontal gap between components to the specified value.
- **setRows(int rw):** Sets the number of rows in this layout to the specified value.
- **setVgap(int vgap):** Sets the vertical gap between components to the specified value.
- **layoutContainer(Container pr):** Lays out the specified container using this layout.

- **toString():** Returns the string representation of this grid layout's values.

Layout Managers (CardLayout)

- The CardLayout class manages the components in such a way that only one component is visible at a time. It treats each component as a card in the container. Only one card is visible at a time, and the container acts as a stack of cards. The first component added to a *CardLayout* object is the visible component when the container is first displayed.

Constructors:

- **CardLayout():** It is used to create a new card layout with gaps of size is zero.
- **CardLayout(int horizontalgap, int verticalgap):** It is used to create a new CardLayout class with the specified horizontal and vertical gaps.

Layout Managers (CardLayout)

Commonly Used Methods:

•**getLayoutAlignmentX(Container parent):** Returns the alignment along the x-axis.

•**getLayoutAlignmentY(Container parent):** Returns the alignment along the y-axis.

•**getVgap():** Used to get the vertical gap between components.

•**addLayoutComponent(Component cm, Object cn):** Used to add the specified component to this card layout's internal table of names.

•**getHgap():** Used to get the horizontal gap between components.

•**toString():** Returns a string representation of the state of this card layout.

•**removeLayoutComponent(Component cm):** Used to remove the specified component from the layout.

Exercise

Q) Design a registration form using AWT components.

1. Components to include:

   Label, TextField, Checkbox, Choice, Button

2. Initialize all components using constructors.

3. Arrange components using FlowLayout.

JavaBean

Introduction

- *JavaBeans* (the architecture) defines a set of rules;.
- *Java beans* are ordinary Java objects that play by these rules.
- That is, Java beans are Java objects that conform to the JavaBeans API and design patterns.
- By doing so, they can be recognized and manipulated by tools (especially visual application builder environments) as well as by hand coding.
- Beans live and work in the Java runtime system, as do all Java objects.
- They communicate with their neighbors' using events and other normal method invocations.

Introduction

- One of the goals of JavaBeans is to allow components to be manipulated visually within a graphical application builder.
- Beans can be chosen from a palette of tools and manipulated graphically by an application designer.
- But Java beans can be not only simple UI components, such as buttons and sliders, but also more complex and abstract components.
- It is easy to get the impression that beans are, themselves, always graphical objects, but Java beans can implement any part of an application, including “invisible” parts that perform calculations, storage, and communications.

Introduction

- *Three characteristics of the JavaBeans architecture aim to make it possible to work with application components in this way:*
  - ***Design patterns**: The most important characteristic of a Java bean is simply a layer of standardization. Design patterns (i.e., coding conventions) let tools and humans recognize the basic features of a bean and manipulate it without knowing how it is implemented.*
  - ***Reflection** makes it possible for Java code to inspect and manipulate new Java objects at runtime. In the context of JavaBeans, reflection lets a development tool analyze a bean's capabilities, examine the values of its fields, and invoke its methods.*
  - ***Object serialization:** Finally, the Java Serialization API allows us to "freeze-dry" a live application or application component and revive it later. This is an important capability that makes it possible to piece together applications without extensive code generation.*

Introduction

- A JavaBean is a Java class that should follow the following conventions:
  - It should have a no-arg constructor.
  - It should be Serializable.
  - It should provide methods to set and get the values of the properties, known as getter and setter methods.

Why use JavaBean?

- According to Java white paper, it is a reusable software component.
- A bean encapsulates many objects into one object so that we can access this object from multiple places.
- Moreover, it provides easy maintenance.

Simple example of JavaBean class

```
//Employee.java

package mypack;
public class Employee implements java.io.Serializable{
private int id;
private String name;
public Employee(){}
public void setId(int id){this.id=id;}
public int getId(){return id;}
public void setName(String name){this.name=name;}
public String getName(){return name;}
}
```

How to access the JavaBean class?

•To access the JavaBean class, we should use getter and setter methods.

```
package mypack;
public class Test{
public static void main(String args[]){
Employee e=new Employee();//object is created
e.setName("Priscilla");//setting value to the object
System.out.println(e.getName());
}}
```

•Note: There are two ways to provide values to the object. One way is by constructor and second is by setter method.

Implementing Java Bean

```
public class Employee implements java.io.Serializable
{
private int id;
private String name;
public Employee()
  {
  }
public void setId(int id)
  {
    this.id = id;
  }
public int getId()
  {
    return id;
  }
public void setName(String name)
  {
    this.name = name;
  }
public String getName()
  {
    return name;
  }
}
```

Implementing Java Bean

```
public class Employee1 {

public static void main(String args[])
{
    Employee s = new Employee();
    s.setName("Priscilla");
    System.out.println(s.getName());
}
}
```

Output:
Priscilla

JavaBean Properties

- A JavaBean property is a named feature that can be accessed by the user of the object. The feature can be of any Java data type, containing the classes that you define.
- A JavaBean property may be read, write, read-only, or write-only. JavaBean features are accessed through two methods in the JavaBean's implementation class:

JavaBean Properties

1. getPropertyName ()

For example, if the property name is firstName, the method name would be getFirstName() to read that property. This method is called the accessor.

2. setPropertyName ()

For example, if the property name is firstName, the method name would be setFirstName() to write that property. This method is called the mutator.

JavaBean Properties

Properties of setter Methods

- It must be public in nature.
- The return type a should be void.
- The setter method uses prefix set.
- It should take some argument.

Properties of getter Methods

- It must be public in nature.
- The return type should not be void.
- The getter method uses prefix get.
- It does not take any argument.

Advantages of JavaBean

- The following are the advantages of JavaBean:
  - It is portable, compact, and easy.
  - The JavaBean properties and methods can be exposed to another application.
  - It provides an easiness to reuse the software components.

Disadvantages of JavaBean

The following are the disadvantages of JavaBean:

- JavaBeans are mutable. So, it can't take advantages of immutable objects.
- Creating the setter and getter method for each property separately may lead to the boilerplate code.

Benefits of Using Beans in Java

some of the primary benefits of using JavaBeans:

•**Reusability:** JavaBeans are designed with reusability in mind. Once a bean is created, it can be used across different applications. It allows developers to build upon existing components rather than starting from scratch, significantly speeding up the development process.

•**Encapsulation:** JavaBeans encapsulate many objects into one, hiding the internal complexity of individual components from other parts of the program. Encapsulation helps in managing complexity by exposing only necessary parts of the functionality.

Benefits of Using Beans in Java

some of the primary benefits of using JavaBeans:

•**Ease of Use:** It can be manipulated visually in a builder tool. Most IDEs support JavaBeans by providing features like dragging and dropping beans into applications, which makes it easier for non-programmers or developers to build applications.

•**Integration and Flexibility:** JavaBeans can be integrated into different environments and can communicate with other components provided these components adhere to the Beans' conventions. This flexibility also extends to their capability to operate across different operating systems and platforms.

Benefits of Using Beans in Java

some of the primary benefits of using JavaBeans:

•**Customization:** JavaBeans can be customized easily using property editors and customizers. It allows other developers and software tools to configure a bean to fit their specific environment.

•**Persistence:** JavaBeans can be made persistent, meaning their state can be saved to and restored from storage, without the need for the application to understand the beans' internals. This persistence is facilitated by the Serializable interface.

Benefits of Using Beans in Java

some of the primary benefits of using JavaBeans:

•**Security:** Beans provide a security framework that supports features like signing, constraints, and role-based authorization, which can be critical for applications that require high security.

•**Interoperability:** Since JavaBeans components are written in Java, they are inherently interoperable with Java applications. However, with appropriate adapters, JavaBeans can also be made to work with components written in other languages, promoting cross-platform interaction.

Life Cycle of Java Beans

1. **Instantiation**
2. **Customization (Optional)**
3. **Setting Property Values**
4. **Connection with Other Beans**
5. **Activation (Optional)**
6. **Introspection**
7. **Running**
8. **Passivation (Optional)**
9. **Destruction**

Life Cycle of Java Beans

- The life cycle of a JavaBean refers to the various stages it goes through from instantiation to destruction.
- Understanding these stages is crucial for effective bean management, especially in environments where beans are used extensively, such as Java EE or when beans are managed by containers in frameworks like Spring.
- Here's a detailed look at the typical life cycle of a JavaBean:

1. Instantiation

The first stage in the life cycle of a JavaBean is instantiation. During this stage, the JavaBean is created, usually by calling a no-argument constructor. It is a simple creation step where the bean is allocated memory and its initial state is set up.

Life Cycle of Java Beans

2. Customization (Optional)

After a bean is instantiated, it may undergo customization. It involves configuring the bean's properties and other settings to suit specific needs. Customization typically happens in two ways:

- **Design Time:** Using tools such as IDEs where beans can be configured using property sheets.
- **Runtime:** Programmatically setting properties through setter methods or configuration files (XML, JSON, etc.).

3. Setting Property Values

Once the bean is customized, its properties are set. It is often done through setter methods defined in the bean. Property values can be set directly by the application that uses the bean, or by a framework or container that manages the bean.

Life Cycle of Java Beans

4. Connection with Other Beans

JavaBeans can be connected to other beans. It can involve setting up event listeners and event sources, allowing beans to respond to events triggered by other beans. This step is crucial for the bean to interact within an application or environment, supporting the event-driven programming model.

5. Activation (Optional)

For beans that are serialized and later restored, the activation stage involves restoring the bean from its serialized state. Activation may involve re-establishing transient connections or resources that were not saved during serialization.

6. Introspection

Introspection is a process where the capabilities of a bean are examined dynamically. Tools and frameworks can inspect a bean to determine its properties, events, and methods. This allows for dynamic configuration and interaction with the bean.

Life Cycle of Java Beans

7. Running

During this stage, the bean is in use and performing its intended tasks. It may handle data processing, respond to user input, or interact with other components and beans. This phase is generally where the bean spends most of its lifecycle.

8. Passivation (Optional)

In environments where beans are managed (like EJB containers), passivation is a stage where a bean's state is temporarily stored to free resources. It usually happens when the bean is not currently needed, but its state needs to be preserved for later activation.

9. Destruction

The final stage in the lifecycle of a JavaBean is its destruction. Here, the bean is marked for garbage collection, and any resources it used are released. Before destruction, cleanup methods can be called to ensure that all resources are freed properly, such as closing database connections or releasing file handles.

BEAN DEVELOPMENT KIT

- The Beans Development Kit (BDK) is intended to support the early development of JavaBeans™ components and to act as a standard reference base for both bean developers and tool vendors.
- The BDK contains a number of useful things:
  - JavaBean source files
  - The BeanBox
  - JavaBeans demos
  - JavaBeans documentation

JAR File

- As you are aware that *javac.exe* creates a separate **.class** file for every class defined in the program.
- Hence, if a program has five classes, the compiler will create five class file each with their respective name and a **.class** extension.
- This proved to be drawback as the applet loader makes separate connections for each file while loading the applet.
- To overcome this problem, the engineers at Java Soft, the *Sun Microsystems subsidiary for Java* introduced the idea for compressing all the files together which can be unzipped at the applet's client machine when the applet executes.

JAR File

- Jar is a zip or compression utility tailored to Java's needs.
- Pass the names of all the files in the project to the JAR utility, which compresses all of them together and creates a compressed file with a **.jar** extension.

- Usage of JAR is as under:

  jar [-cvfmOM][jar-file name][manifest-file name] list of .class file

- Example:
- jar -cvfm SimpleBean.jar SimpleBean.mf SimpleBean.class

Handling Events in JavaBeans

Introduction

- Enables Beans to communicate and connect together. Beans generate events and these events can be sent to other objects
- Event means any activity that interrupts the current ongoing activity.
- Example: mouse clicks, pressing key…
- User-defined JavaBeans interact with the help of user-defined events, which are also called custom events.
- Can use the Java event delegation model to handle these custom events.

Introduction

- The components of the event delegation model are:

  ✓ Event Source: Generates the event and informs all the event listeners that are registered with it

  ✓ Event Listener: Receives the notification, when an event source generates an event

  ✓ Event Object: Represents the various types of events that can be generated by the event sources.

Creating Custom Events

- The classes and interfaces that you need to define to create the custom JavaBean events are:
  - An **event class** to define a custom JavaBean event.
  - An **event listener interface** for the custom JavaBean event.
  - An **event handler** to process the custom JavaBean event.
- A target Java application that implements the custom event.

Creating the Event Class

- The event class that defines the custom event extends the EventObject class of the java.util package.

•***Example***

```
public class NumberEvent extends EventObject
{
Public int number1,number2;
Public NumberEvent(Object o, int number1, int number2)
{
super(o); this.number1=number1;
this.number2=number2;
} }
```

- Beans can generate events and send them to other objects.

Creating Event Listeners

- When the event source triggers an event, it sends a notification to the event listener interface.
- **The event listener interface implements the java.util.EventListener interface.**

  Syntax

  - public void addTListener(TListenereventListener);
  - public void addTListener(TListenereventListener)throws TooManyListeners;
  - public void removeTListener(TListenereventListener);
- The target application that uses the custom event implements the custom listener.

Example

- public interface NumberEnteredListener extends EventListener{
- public void arithmeticPerformed(NumberEventmec); }

Creating Event Handler

- Custom event handlers should define the following methods:
  - addXXListener(): Registers listeners of a JavaBean event.
  - fireXX(): Notifies the listeners of the occurrence of a JavaBean event.
  - removeXXListener(): Removes a listener from the list of registered listeners of a JavaBean.

- ***The code snippet to define an event handler for the custom event NumberEvent***

```
public class NumberBean extends JPanel implements ActionListener{
Public NumberBean(){}
NumberEnteredListenermel;
public void addNumberListener(NumberEnteredListenermel){
this.mel = mel; } }
```

Creating Event Handler

- The Java ActionListener is notified whenever you click on the button or menu item.
- It is notified against ActionEvent.
- The ActionListener interface is found in java.awt.event package.
- It has only one method: actionPerformed().

```
public abstract void actionPerformed(ActionEvent e);
```

Creating Event Handler

- How to write ActionListener:
- 1) **Implement the ActionListener interface in the class:**

```
public class ActionListenerExample Implements ActionListener
```

- 2) **Register the component with the Listener:**

```
component.addActionListener(instanceOfListenerclass);
```

- 3) **Override the actionPerformed() method:**

```
public void actionPerformed(ActionEvent e){
        //Write the code here
}
```

Creating Event Handler

- Java ActionListener Example:

```
import java.awt.*;
import java.awt.event.*;
public class ActionListenerExample {
public static void main(String[] args) {
   Frame f=new Frame("ActionListener Example");
   final TextField tf=new TextField();
   tf.setBounds(50,50, 150,20);
   Button b=new Button("Click Here");
   b.setBounds(50,100,60,30);

   b.addActionListener(new ActionListener(){
   public void actionPerformed(ActionEvent e){
         tf.setText("Welcome to Javatpoint.");

   }
   });
```

Creating Event Handler

```
    f.add(b);
   f.add(tf);
    f.setSize(400,400);
    f.setLayout(null);
    f.setVisible(true);
  }
  }
```

Exercise

Q) **Responsive Calculator Interface Design**

Design a **GUI-based Calculator Application** in Java

Requirements:

1. The calculator must include:
   - Numeric buttons (0–9)
   - Arithmetic operators (+, –, ×, ÷)
   - Special buttons (C, =)
   - A display text field at the top
2. All buttons must be arranged in a **uniform grid structure**.
3. The display field must span across the top of the interface.

Challenge: How would you modify the layout to support portrait and landscape layouts?

Exercise

Q) Develop a JavaBean named ***StockPriceBean*** that:

- Maintains a stock price value.
- Fires a custom event *PriceChangeEvent* whenever price changes significantly.

Tasks:

- Create:
  - Custom event class (extends *EventObject*)
  - Event listener interface
- Implement event registration methods:
  - *addPriceChangeListener()*
  - *removePriceChangeListener()*
- Demonstrate event firing in a test class.

Exercise

Q) You are developing a **Java-based Online Voting System** where eligibility criteria require that a voter must be at least 18 years old.

Tasks:

a) Design and implement a **custom exception class** named *InvalidAgeException* to handle cases where a user does not meet the minimum age requirement.

b) Develop a method *checkAge(int age)* that validates the voter's age.

The method should use the throw keyword to raise the custom exception if the age is below 18.

c) In the main() method, demonstrate how the *checkAge()* method is invoked and how the exception is handled using an appropriate try-catch block.

d) Critically analyze why defining a **custom exception** *(InvalidAgeException)* is more appropriate in this scenario than using a standard exception such as *IllegalArgumentException*.

Exercise

Q) Create a visual JavaBean component named ***ColorSwitcherBean*** that:

- Extends ***JPanel***
- Changes background color when clicked.
- Fires a custom event when color changes.

Tasks:

- Add property ***currentColor***
- Implement custom event ***ColorChangeEvent***
- Demonstrate how the Bean can be visually manipulated in a Bean development environment.

END